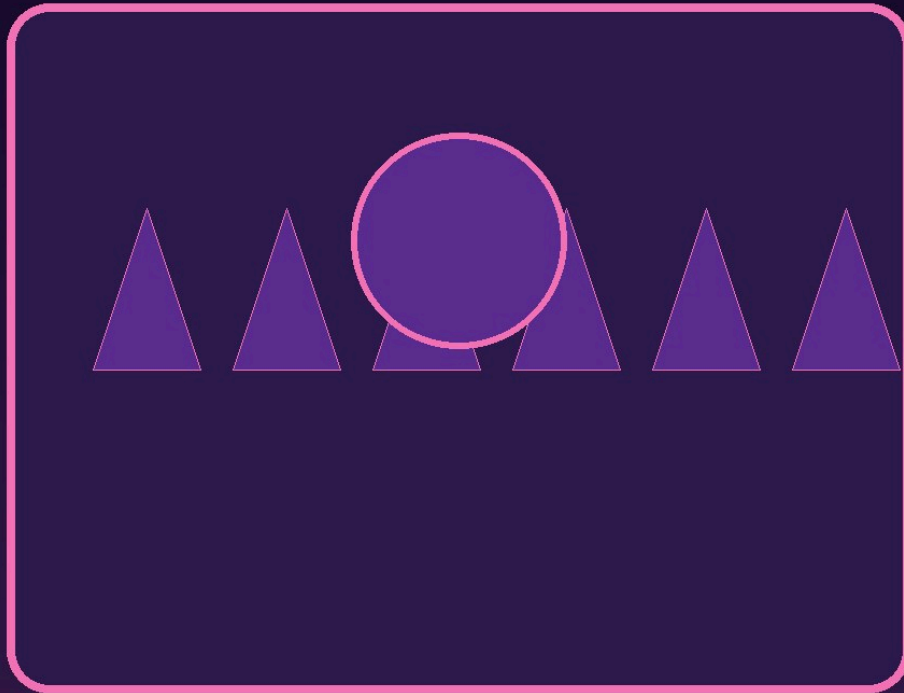




FORMATION KOTLIN - LES BASES

Kotlin

Apprendre le langage moderne Android et
JVM, pas a pas.

DanielCraft - Livre debutant - Pack Mobile & JVM

Sommaire

Clique un chapitre ou un sous-chapitre pour y aller.

1. C'est quoi Kotlin ?

- Le langage en une phrase
- Pourquoi apprendre Kotlin ?
- A quoi ca ressemble ?
- Petite histoire
- En vrai
- A retenir

2. Installer Kotlin

- Ce qu'il te faut
- Installation rapide
- Gradle (projets Android / backend).
- Petite histoire
- A retenir

3. Premier programme

- Hello World
- Affichage formate
- Les commentaires
- Petite histoire
- A retenir

4. Les variables

- val vs var
- Types explicites
- Constantes
- Conventions
- Petite histoire
- A retenir

5. Les types de donnees

- Types numeriques
- Texte et booleen
- Conversion
- Nullable types
- Petite histoire
- A retenir

6. Les conditions

- if / else
- if comme expression
- when (equivalent switch ameliore).
- Petite histoire
- A retenir

7. Les boucles

- [for et ranges](#)
- [Parcourir une collection](#)
- [while et do-while](#)
- [break et continue](#)
- [Petite histoire](#)
- [A retenir](#)

8. [Les fonctions](#)

- [Declarer une fonction](#)
- [Fonction expression \(corps unique\)](#)
- [Parametres par default](#)
- [Fonctions lambda](#)
- [Petite histoire](#)
- [A retenir](#)

9. [Null safety](#)

- [Le probleme du null](#)
- [Operateur safe call](#)
- [Elvis operator](#)
- [Force unwrap \(a eviter\)](#)
- [Smart cast](#)
- [Petite histoire](#)
- [A retenir](#)

10. [Classes et objets](#)

- [Definir une classe](#)
- [Proprietes calculees](#)
- [Visibilite](#)
- [Object \(singleton\)](#)
- [Petite histoire](#)
- [A retenir](#)

11. [Data classes](#)

- [Le probleme](#)
- [Destructuration](#)
- [copy\(\)](#)
- [Quand utiliser data class](#)
- [Petite histoire](#)
- [A retenir](#)

12. [Collections](#)

- [list, set, map](#)
- [Operations courantes](#)
- [Parcourir une map](#)
- [Sealed classes \(bonus\)](#)
- [Petite histoire](#)
- [A retenir](#)

13. [Gerer les erreurs](#)

- [try / catch](#)
- [Result<T> \(idiome Kotlin\)](#)
- [require et check](#)
- [runCatching](#)
- [Petite histoire](#)
- [A retenir](#)

14. [Mini-projet : liste de courses CLI](#)

- [L'objectif](#)
- [Structure](#)
- [Ce que tu apprends](#)
- [A retenir](#)

15. [Ce qu'il faut retenir](#)

- [Les briques essentielles](#)
- [La methode](#)
- [Ce qui vient apres](#)
- [A retenir](#)

16. [Atelier : variables et types](#)

- [Exercice 1 : carte d'identite](#)
- [Exercice 2 : conversion](#)
- [Exercice 3 : nullable](#)
- [Defi : temperature](#)
- [A retenir](#)

17. [Atelier : fonctions](#)

- [Exercice 1 : salutation](#)
- [Exercice 2 : moyenne](#)
- [Exercice 3 : filtrage](#)
- [Exercice 4 : division securisee](#)
- [A retenir](#)

18. [Erreurs classiques en Kotlin](#)

- [1. Utiliser var au lieu de val](#)
- [2. Force unwrap avec !!](#)
- [3. Oublier le ? pour nullable](#)
- [4. Confondre listOf et mutableListOf](#)
- [5. Oublier le return dans une fonction bloc](#)
- [6. Smart cast impossible](#)
- [A retenir](#)

19. [Bonnes pratiques](#)

- [Conventions Kotlin](#)
- [Idiomes Kotlin](#)
- [Null safety](#)
- [Tests](#)
- [A retenir](#)

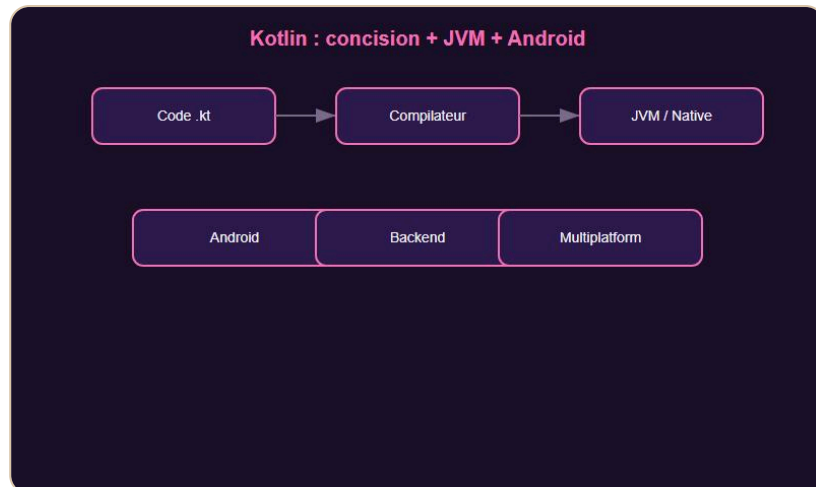
20. [Quiz Kotlin - Les bases](#)

- [Teste tes connaissances](#)
- [Reponses](#)

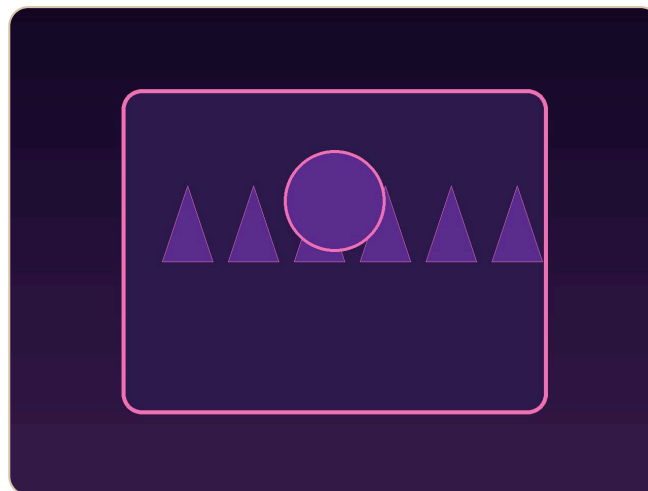
21. [Bravo !](#)

- [Tu as termine Kotlin - Les bases](#)
- [Ce que tu as construit](#)
- [La suite avec DanielCraft](#)
- [Un dernier conseil](#)

C'est quoi Kotlin ?



Kotlin : concision + JVM + Android.



Couverture Kotlin.

Le langage en une phrase

Kotlin est un langage moderne cree par JetBrains en 2011. Il tourne sur la JVM (comme Java), compile en natif et en JavaScript. Google l'a declare langage officiel pour Android en 2017.

Pourquoi apprendre Kotlin ?

Kotlin est concis, sur (null safety integree) et interoperable avec Java. Il est utilise par Google, Netflix, Pinterest, Uber et des millions d'apps Android. Stack Overflow le classe regulierement parmi les langages les plus aimes.

> **Astuce DanielCraft** - Si tu vises Android ou le backend JVM, Kotlin est le choix moderne par excellence.

Nora decouvre Kotlin en voulant ecrire moins de code que Java pour la meme fonctionnalite. Elle cree une classe en 5 lignes au lieu de 20.

A quoi ca ressemble ?

```
fun main() {  
    val nom = "Lea"  
    println("Salut $nom, bienvenue en Kotlin !")  
}
```

Petite histoire

JetBrains cree Kotlin pour corriger les frustrations de Java : verbosite, null pointer exceptions, absence de lambdas modernes. Le langage se stabilise en 2016 et devient le standard Android.

En vrai

Sam developpe une app Android avec Jetpack Compose. Max migre un backend Spring de Java vers Kotlin. Nora ecrit un script CLI en 30 lignes.

> **Piege** - Kotlin n'est pas du Java raccourci. Il a ses idiomes : `val`, `data class`, `when`, null safety.

A retenir

- Kotlin = concision + securite + interop Java.
- Langage officiel Android.
- `val` immutable, `var` mutable.

Installer Kotlin



IntelliJ + Gradle : pret en minutes.

Ce qu'il te faut

- **IntelliJ IDEA** (Community gratuit) ou Android Studio pour le mobile.
- Le compilateur Kotlin est inclus dans IntelliJ.

Installation rapide

1. 1. Telecharge IntelliJ IDEA sur jetbrains.com/idea.
2. 2. Installe le plugin Kotlin (pre-installe dans les versions recentes).
3. 3. File > New > Project > Kotlin > JVM.

Verification :

```
kotlinc -version
```

Gradle (projets Android / backend)

La plupart des projets Kotlin utilisent Gradle :

```
// build.gradle.kts
plugins {
    kotlin("jvm") version "2.0.0"
}
```

> **Astuce DanielCraft** - IntelliJ + Kotlin = auto-completion, refactoring et erreurs en temps reel. C'est l'environnement ideal.

Petite histoire

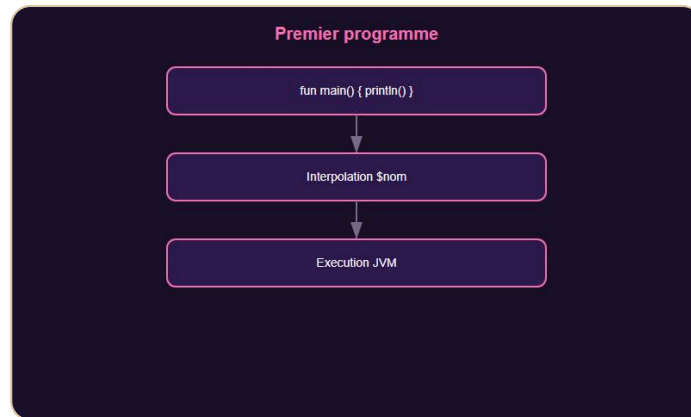
Max installe IntelliJ, cree un projet Kotlin JVM, tape `fun main()` et lance. Le programme s'execute en un clic.

A retenir

- IntelliJ IDEA = IDE de reference.
- Android Studio pour le mobile.

- Gradle pour les projets professionnels.

Premier programme



`fun main()` et `println` : premier programme.

Hello World

```
fun main() {
    println("Bonjour le monde !")
}
```

`fun` declare une fonction. `main()` est le point d'entree.

Affichage formate

```
val nom = "Sam"
val age = 28
println("Je suis $nom, $age ans.")
println("${nom.uppercase()} a $age ans.")
```

Les `$` et `${}` permettent l'interpolation de chaines.

Les commentaires

```
// Commentaire sur une ligne
/* Commentaire
sur plusieurs lignes */
```

Petite histoire

Nora cree un fichier `Main.kt`, ecrit 3 lignes, clique Run. Le message s'affiche. Kotlin est concis des le depart.

A retenir

- `fun main()` = point d'entree.
- `println()` pour afficher.
- Interpolation avec `$variable` ou `${expression}`.

Les variables



val immutable, var mutable.

val vs var

```
val nom = "Max"           // Immutable (comme final en Java)
var score = 0             // Mutable
score += 10
println(score)            // 10
```

> **Astuce DanielCraft** - Prefere `val` par default. Utilise `var` seulement quand la valeur doit changer.

Types explicites

```
val age: Int = 25
val prix: Double = 19.99
val actif: Boolean = true
```

Kotlin infere le type quand c'est evident :

```
val ville = "Paris"      // String infere
val compteur = 42        // Int infere
```

Constantes

```
const val TVA = 0.20
const val MAX_JOUEURS = 100
```

`const val` doit etre un type primitif ou String, declare au niveau top-level ou dans un objet.

Conventions

- camelCase pour variables et fonctions : `monScore` .
- PascalCase pour classes : `MonType` .
- SCREAMING_SNAKE_CASE pour constantes : `MAX_JOUEURS` .

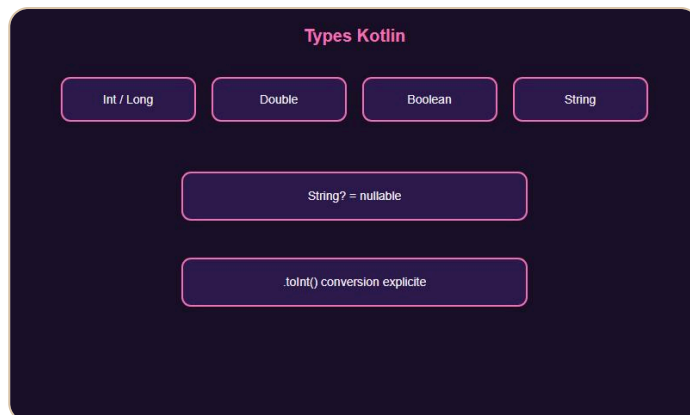
Petite histoire

Sam declare `val budget = 1800` puis essaie `budget = 1500`. Le compilateur refuse. Il comprend : `val` = immutable.

A retenir

- `val` = immutable, `var` = mutable.
- Inference de type par défaut.
- `const val` pour les constantes compile-time.

Les types de donnees



Int, Double, String, nullable ?.

Types numeriques

Type	Exemple	Usage
Int	42	Entier (default)
Long	9_000_000_000L	Grand entier
Double	3.14	Flottant (default)
Float	3.14f	Flottant simple precision

Texte et booleen

```
val texte: String = "Bonjour"
val lettre: Char = 'A'
val ok: Boolean = true
```

Conversion

```
val i = "42".toInt()
val s = 42.toString()
val d = 3.14.toInt() // 3
```

Nullable types

```
var nom: String? = null // Peut etre null
nom = "Lea"
```

Le `?` indique qu'une variable peut etre null. C'est le coeur de la null safety Kotlin.

> **Astuce DanielCraft** - Pas de conversion implicite dangereuse. Chaque conversion est explicite avec `.toInt()`, `.toString()`, etc.

Petite histoire

Nora recoit une chaine "42" et veut la multiplier par 2. Elle appelle `.toInt()` et obtient 84. Pas de magie, tout est explicite.

A retenir

- `Int`, `Double`, `String`, `Boolean` sont les types courants.
- `String?` = nullable, `String` = jamais null.
- Conversions explicites avec `.toXxx()`.

Les conditions



if expression + when.

if / else

```
val age = 17
if (age >= 18) {
    println("Majeur")
} else {
    println("Mineur")
}
```

if comme expression

```
val statut = if (age >= 18) "Majeur" else "Mineur"
println(statut)
```

En Kotlin, `if` retourne une valeur. Pas besoin de ternaire `? : .`

when (equivalent switch ameliore)

```
val jour = "lundi"
when (jour) {
    "lundi", "mardi" -> println("Debut de semaine")
    "vendredi" -> println("Presque le weekend")
    else -> println("Autre jour")
}
```

`when` peut aussi matcher des types, des ranges, des conditions :

```
when {
    age < 12 -> println("Enfant")
    age < 18 -> println("Adolescent")
    else -> println("Adulte")
}
```

> **Astuce DanielCraft** - `when` remplace `switch` et fait bien plus. C'est l'idiome Kotlin pour les conditions multiples.

Petite histoire

Max écrit un `when` sur une enum `Direction`. Le compilateur vérifie l'exhaustivité. Il ajoute le cas manquant et le code compile.

A retenir

- `if` est une expression (retourne une valeur).
- `when` remplace `switch`, plus puissant.
- Pas de ternaire : utilise `if` expression.

Les boucles



for, while, forEach.

for et ranges

```
for (i in 0..4) {  
    println(i) // 0, 1, 2, 3, 4  
}  
  
for (i in 0 until 5) {  
    println(i) // 0, 1, 2, 3, 4 (5 exclu)  
}  
  
for (i in 5 downTo 1 step 2) {  
    println(i) // 5, 3, 1  
}
```

Parcourir une collection

```
val fruits = listOf("pomme", "banane", "cerise")  
for (fruit in fruits) {  
    println(fruit)  
}  
  
fruits.forEach { println(it) }
```

while et do-while

```
var n = 0  
while (n < 5) {  
    println(n)  
    n++  
}
```

break et continue

```
for (i in 0..10) {  
    if (i == 5) break  
    if (i % 2 == 0) continue  
    println(i) // 1, 3  
}
```

> **Astuce DanielCraft** - `for (x in collection)` est l'idiome principal. Evite les index manuels quand possible.



Coder en Kotlin au quotidien.

Petite histoire

Sam utilise `for (i in 1..100)` pour calculer une somme. Pas de variable d'index separee, pas d'oubli d'incrementation.

A retenir

- `0..4` = range inclusif, `0 until 5` = exclusif.
- `for (x in liste)` pour parcourir.
- `forEach { }` pour les lambdas.

Les fonctions



Fonctions et lambdas.

Declarer une fonction

```
fun saluer(prenom: String) {
    println("Bonjour $prenom !")
}

fun additionner(a: Int, b: Int): Int {
    return a + b
}
```

Fonction expression (corps unique)

```
fun doubler(x: Int) = x * 2

fun estMajeur(age: Int) = age >= 18
```

Pas de `{ }` ni de `return` : le `=` suffit.

Parametres par default

```
fun saluer(prenom: String, message: String = "Bonjour") {
    println("$message $prenom !")
}

saluer("Lea")                // Bonjour Lea !
saluer("Lea", "Salut")       // Salut Lea !
```

Fonctions lambda

```
val nombres = listOf(3, 1, 4, 1, 5)
val doubles = nombres.map { it * 2 }
val pairs = nombres.filter { it % 2 == 0 }
```

> **Astuce DanielCraft** - `it` est le nom implicite du parametre dans une lambda a un seul argument.

Petite histoire

Max écrit `fun calculerTTC(prix: Double) = prix * 1.20`. Une ligne, pas de `{ }`, pas de `return`. Kotlin est concis.

A retenir

- `fun nom(params): TypeRetour { }`
- `= expression` pour les fonctions courtes.
- Paramètres par défaut évitent la surcharge.
- Lambdas avec `{ it -> ... }` ou `{ ... }`.

Null safety



Null safety : le coeur de Kotlin.

Le probleme du null

En Java, `NullPointerException` est la cause numero 1 de crash. Kotlin elimine ce risque a la compilation.

```
var nom: String = "Lea"    // Ne peut PAS etre null
var alias: String? = null // Peut etre null
```

Operateur safe call

```
val longueur = alias?.length // null si alias est null
println(longueur)           // null (pas de crash)
```

Elvis operator

```
val longueur = alias?.length ?: 0
// Si alias est null, retourne 0
```

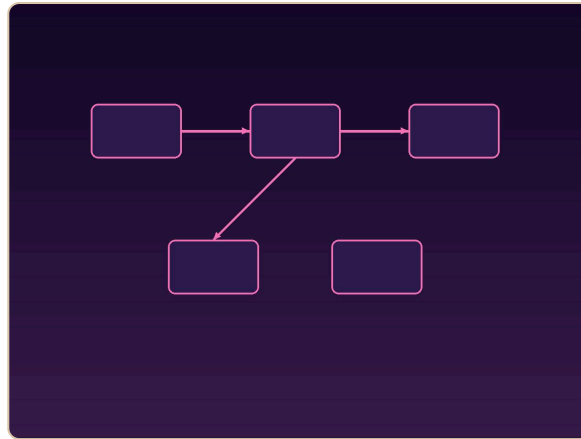
Force unwrap (a eviter)

```
val longueur = alias!!.length // Crash si null !
```

> **Piege** - `!!` force le deballage. Utilise-le seulement quand tu es certain que la valeur n'est pas null.

Smart cast

```
fun afficherLongueur(texte: String?) {
    if (texte != null) {
        println(texte.length) // Kotlin sait que texte n'est plus null
    }
}
```



Visualiser ?. et ?:

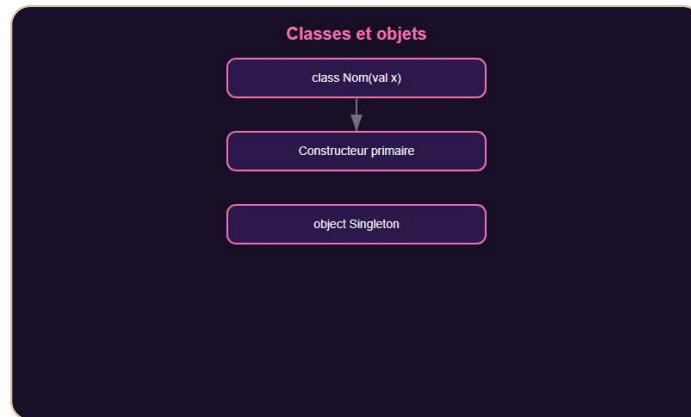
Petite histoire

Nora recoit un champ nullable d'une API. Au lieu de crasher, elle utilise `?.` et `?: "Inconnu"`. Plus jamais de `NullPointerException`.

A retenir

- `String` = jamais null, `String?` = peut être null.
- `?.` = safe call, `?:` = valeur par défaut.
- Évitez `!!` en production.

Classes et objets



Classes et objets.

Définir une classe

```

class Animal(val nom: String, var age: Int) {
    fun sePresenter() {
        println("Je suis $nom, $age ans.")
    }
}

val chat = Animal("Felix", 3)
chat.sePresenter()
  
```

Les paramètres du constructeur primaire deviennent automatiquement propriétés.

Propriétés calculées

```

class Rectangle(val largeur: Int, val hauteur: Int) {
    val aire: Int
        get() = largeur * hauteur
}
  
```

Visibilité

```

class Compte(private var solde: Double) {
    fun depot(montant: Double) { solde += montant }
    fun lireSolde() = solde
}
  
```

Object (singleton)

```

object Config {
    const val VERSION = "1.0"
    val API_URL = "https://api.example.com"
}

println(Config.VERSION)
  
```

> **Astuce DanielCraft** - Le constructeur primaire avec `val` / `var` remplace le boilerplate Java.

Petite histoire

Max cree une classe `Produit` en 3 lignes avec constructeur primaire. En Java, il aurait eu 15 lignes de getters/setters.

A retenir

- `class Nom(val x: Type)` = constructeur + propriete.
- `get()` pour les proprietes calculees.
- `object` pour les singletons.

Data classes



Data classes.

Le probleme

En Java, une simple classe de donnees necessite constructeur, getters, equals, hashCode, toString. En Kotlin :

```

data class Produit(val nom: String, val prix: Double)

val p1 = Produit("Clavier", 49.99)
val p2 = Produit("Clavier", 49.99)
println(p1)           // Produit(nom=Clavier, prix=49.99)
println(p1 == p2)     // true
  
```

Destructuration

```

val (nom, prix) = p1
println("$nom coute $prix EUR")
  
```

copy()

```

val p3 = p1.copy(prix = 39.99)
println(p3) // Produit(nom=Clavier, prix=39.99)
  
```

Quand utiliser data class

- Modeles de donnees (DTO, entites).
- Resultats de fonctions avec plusieurs valeurs.
- Toute classe dont l'egalite est basee sur le contenu.

> **Astuce DanielCraft** - `data class` genere automatiquement equals, hashCode, toString, copy et componentN. Indispensable.

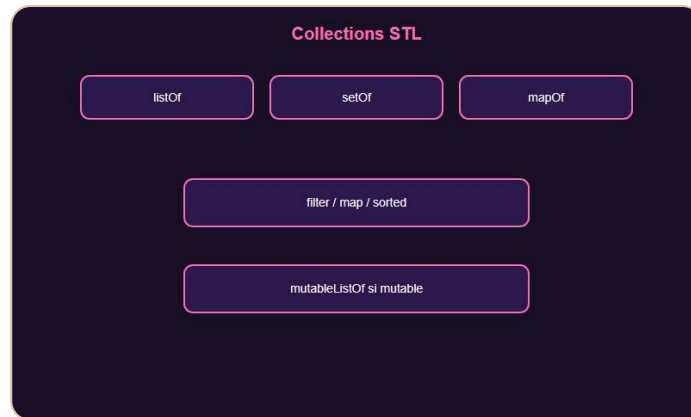
Petite histoire

Nora remplace 40 lignes de Java par `data class User(val id: Int, val name: String, val email: String)`. Le compilateur fait le reste.

A retenir

- `data class` = equals + hashCode + toString + copy auto.
- Destructuration avec `val (a, b) = objet`.
- `.copy(champ = nouvelleValeur)` pour cloner avec modification.

Collections



listOf, filter, map.

list, set, map

```

val fruits = listOf("pomme", "banane", "cerise")    // Immutable
val notes = mutableListOf(12, 15, 18)               // Mutable
val unique = setOf(1, 2, 3, 2, 1)                   // {1, 2, 3}
val ages = mapOf("Lea" to 28, "Sam" to 22)          // Immutable
  
```

Operations courantes

```

val nombres = listOf(3, 1, 4, 1, 5, 9)

nombres.size           // 6
nombres.sorted()       // [1, 1, 3, 4, 5, 9]
nombres.filter { it > 3 } // [4, 5, 9]
nombres.map { it * 2 }   // [6, 2, 8, 2, 10, 18]
nombres.sum()           // 23
nombres.maxOrNull()     // 9
  
```

Parcourir une map

```

for ((nom, age) in ages) {
    println("$nom a $age ans")
}
  
```

Sealed classes (bonus)

```

sealed class Resultat {
    data class Succes(val data: String) : Resultat()
    data class Erreur(val message: String) : Resultat()
}
  
```

> **Astuce DanielCraft** - Prefere `listOf` (immutable) par default. Utilise `mutableListOf` seulement si tu dois modifier.

Petite histoire

Sam filtre une liste de 1000 produits avec `.filter { it.prix < 50 }`. Une ligne, pas de boucle manuelle.

A retenir

- `listOf`, `setOf`, `mapOf` = immutables.
- `filter`, `map`, `sorted`, `sum` = opérations fonctionnelles.
- `for ((k, v) in map)` pour parcourir une map.

Gérer les erreurs



Result et runCatching.

try / catch

```

try {
    val n = "abc".toInt()
} catch (e: NumberFormatException) {
    println("Pas un nombre : ${e.message}")
}
  
```

Result<T> (idiome Kotlin)

```

fun diviser(a: Int, b: Int): Result<Int> {
    return if (b == 0) Result.failure(IllegalArgumentException("Division par zero"))
    else Result.success(a / b)
}

diviser(10, 2).onSuccess { println("Resultat : $it") }
diviser(10, 0).onFailure { println("Erreur : ${it.message}") }
  
```

require et check

```

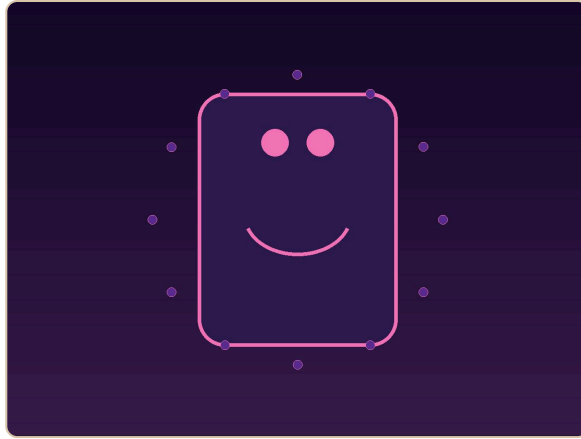
fun retirer(solde: Double, montant: Double): Double {
    require(montant > 0) { "Montant doit etre positif" }
    check(montant <= solde) { "Solde insuffisant" }
    return solde - montant
}
  
```

runCatching

```

val resultat = runCatching { "42".toInt() }
println(resultat.getOrElse(0)) // 42
  
```

> **Astuce DanielCraft** - `Result<T>` est préféré aux exceptions pour les erreurs attendues. Réserve `throw` aux cas vraiment exceptionnels.



Kotlin pour Android.

Petite histoire

Max refactorise un code Java avec 5 try/catch en Kotlin avec `Result` et `runCatching`. Le code est plus lisible et plus sûr.

A retenir

- `try/catch` pour les exceptions.
- `Result<T>` pour les erreurs attendues.
- `require` et `check` pour valider les préconditions.

Mini-projet : liste de courses CLI



Mini-projet : liste de courses CLI.

L'objectif

Créer un programme en ligne de commande qui permet d'ajouter, lister et supprimer des articles d'une liste de courses.

Structure

```

fun main() {
    val courses = mutableListOf<String>()

    while (true) {
        println("\n1. Ajouter  2. Lister  3. Supprimer  4. Quitter")
        when (readln().trim()) {
            "1" -> {
                print("Article : ")
                val article = readln().trim()
                if (article.isNotEmpty()) {
                    courses.add(article)
                    println("Ajoute.")
                }
            }
            "2" -> {
                if (courses.isEmpty()) println("Liste vide.")
                else courses.forEachIndexed { i, a -> println("  ${i + 1}. $a") }
            }
            "3" -> {
                print("Numero : ")
                val idx = readln().toIntOrNull()?.minus(1)
                if (idx != null && idx in courses.indices) {
                    courses.removeAt(idx)
                    println("Supprime.")
                }
            }
            "4" -> { println("A bientôt !"); break }
        }
    }
}
  
```

Ce que tu apprends

- `mutableListOf` pour une collection modifiable.

- `when` pour le menu.
- `readln()` pour lire l'entree utilisateur.
- Null safety avec `toIntOrNull()`.

> **Astuce DanielCraft** - Commence par le menu, puis ajoute une commande a la fois.

A retenir

- Collections mutables + `when` + `readln` = CLI complet.
- `toIntOrNull()` evite les crash sur entree invalide.

Ce qu'il faut retenir



Carte des notions Kotlin.

Les briques essentielles

Concept	Resume
Variables	<code>val</code> immutable, <code>var</code> mutable
Types	<code>Int</code> , <code>Double</code> , <code>String</code> , <code>Boolean</code> , nullable <code>?</code>
Conditions	<code>if</code> expression, <code>when</code> puissant
Boucles	<code>for (x in range)</code> , <code>forEach</code>
Fonctions	<code>fun</code> , <code>= expression</code> , lambdas
Null safety	<code>?.</code> , <code>?:</code> , éviter <code>!!</code>
Classes	Constructeur primaire, <code>data class</code>
Collections	<code>listOf</code> , <code>filter</code> , <code>map</code>
Erreurs	<code>Result</code> , <code>runCatching</code> , <code>require</code>

La methode

1. Lis le chapitre.
 2. Tape les exemples dans IntelliJ.
 3. Fais compiler (le compilateur enseigne).
 4. Fais les ateliers.
 5. Reviens quand tu bloques.
- > **Astuce DanielCraft** - Kotlin est concis mais exigeant. Chaque ligne compte.

Ce qui vient apres

- Coroutines (async/await natif).

- Jetpack Compose (UI Android declarative).
- Spring Boot avec Kotlin.
- Multiplatform (KMP).

A retenir

- Kotlin = concision + securite + interop Java.
- `val`, `data class`, `when`, null safety sont les piliers.

Atelier : variables et types



Atelier : variables.

Exercice 1 : carte d'identite

```
val prenom = "Sam"
val age = 22
val ville = "Nantes"
println("Je suis $prenom, $age ans, je vis a $ville.")
```

Exercice 2 : conversion

```
val texte = "42"
val nombre = texte.toInt()
val flottant = nombre.toDouble()
println(flottant) // 42.0
```

Exercice 3 : nullable

```
var pseudo: String? = null
pseudo = "DevNora"
val affichage = pseudo ?: "Anonyme"
println(affichage)
```

Defi : temperature

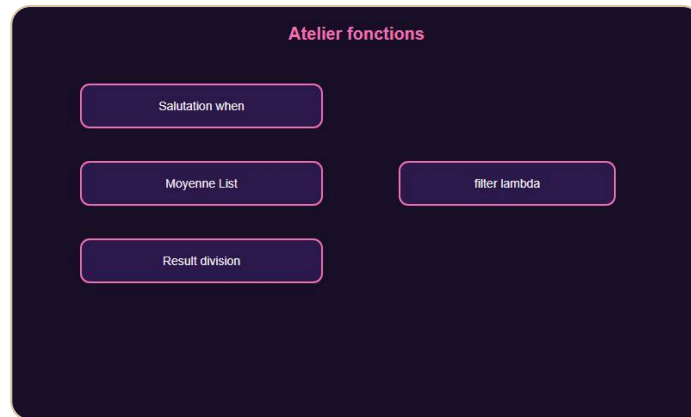
```
val celsius = 37.0
val fahrenheit = celsius * 9 / 5 + 32
println("$celsius°C = $fahrenheit°F")
```

> **Astuce DanielCraft** - Utilise des noms clairs. `celsius` vaut mieux que `c`.

A retenir

- `val` pour immutable, `var` pour mutable.
- `.toInt()`, `.toDouble()` pour les conversions.
- `?: "defaut"` pour les valeurs nullable.

Atelier : fonctions



Atelier : fonctions.

Exercice 1 : salutation

```
fun saluer(nom: String, heure: Int): String = when {
    heure < 12 -> "Bonjour $nom !"
    heure < 18 -> "Bon apres-midi $nom !"
    else -> "Bonsoir $nom !"
}
```

Exercice 2 : moyenne

```
fun moyenne(notes: List<Int>): Double {
    if (notes.isEmpty()) return 0.0
    return notes.average()
}
```

Exercice 3 : filtrage

```
fun motsLongs(mots: List<String>, min: Int) =
    mots.filter { it.length >= min }
```

Exercice 4 : division securisee

```
fun diviser(a: Int, b: Int): Result<Int> =
    if (b == 0) Result.failure(IllegalArgumentException("Division par zero"))
    else Result.success(a / b)
```

> **Astuce DanielCraft** - `List<T>` est prefere a `MutableList<T>` en parametre (plus flexible).

A retenir

- `when` dans les fonctions expression.
- `List<T>` en parametre, pas `MutableList`.
- `Result<T>` pour les erreurs.

Erreurs classiques en Kotlin



Pieges classiques Kotlin.

1. Utiliser var au lieu de val

```
var nom = "Lea" // Devrait etre val si ca ne change pas
```

2. Force unwrap avec !!

```
val longueur = nom!!.length // Crash si null !
```

Utilise `?.` et `?:` a la place.

3. Oublier le ? pour nullable

```
var email: String = null // Erreur de compilation !
var email: String? = null // OK
```

4. Confondre listOf et mutableListOf

```
val liste = listOf(1, 2, 3)
liste.add(4) // Erreur ! listOf est immutable
```

5. Oublier le return dans une fonction bloc

```
fun doubler(x: Int): Int {
    x * 2 // Oublie return !
}
fun doubler(x: Int) = x * 2 // OK avec =
```

6. Smart cast impossible

```
var texte: String? = "hello"
if (texte != null) {
    // texte peut etre modifie par un autre thread
    println(texte.length) // Erreur si var !
}
```

> **Astuce DanielCraft** - Le compilateur Kotlin donne des messages clairs. Lis-les attentivement.

A retenir

- Prefere `val` et `?./?:`.
- `listOf` = immutable, `mutableListOf` = mutable.
- `= expression` evite les oublis de return.

Bonnes pratiques



Bonnes pratiques idiomatiques.

Conventions Kotlin

- **camelCase** : variables, fonctions, propriétés.
- **PascalCase** : classes, interfaces, objects.
- **SCREAMING_SNAKE_CASE** : constantes top-level.
- **ktlint** : formateur/linter officiel.

Idiomes Kotlin

```
// Prefere ceci :
val noms = list.filter { it.actif }.map { it.nom }

// Plutot que :
val noms = mutableListOf<String>()
for (item in list) {
    if (item.actif) noms.add(item.nom)
}
```

Null safety

```
// Prefere :
val nom = utilisateur?.nom ?: "Inconnu"

// Plutot que :
val nom = utilisateur!!.nom
```

Tests

```
@Test
fun `addition de deux nombres positifs`() {
    assertEquals(5, additionner(2, 3))
}
```

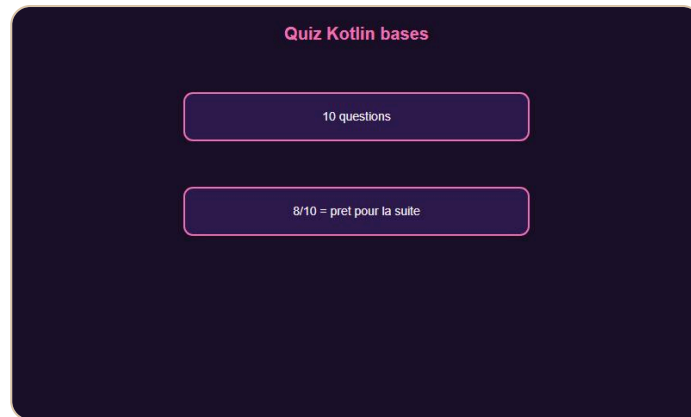
Les noms de test avec backticks sont autorisés en Kotlin.

> **Astuce DanielCraft** - Ecris du Kotlin idiomatique, pas du Java traduit. Utilise `when`, `data class`, lambdas.

A retenir

- Code fonctionnel avec `filter`, `map`, `forEach`.
- Null safety systematique.
- Tests avec noms descriptifs.

Quiz Kotlin - Les bases



Quiz Kotlin bases.

Teste tes connaissances

Q1. Les variables immutables en Kotlin se déclarent avec :

A) var B) val C) const D) let

Q2. Quel opérateur gère le null en Kotlin ?

A) ?? B) ?: C) ?! D) !!

Q3. `data class` génère automatiquement :

A) Un constructeur seulement B) equals, hashCode, toString, copy C) Un singleton D) Une interface

Q4. `when` en Kotlin remplace :

A) for B) while C) switch D) try

Q5. `String?` signifie :

A) String obligatoire B) String nullable C) String vide D) String constante

Q6. Quelle collection est immuable par défaut ?

A) mutableListOf B) ArrayList C) listOf D) MutableList

Q7. `fun doubler(x: Int) = x * 2` est :

A) Une erreur de syntaxe B) Une fonction expression C) Une lambda D) Une constante

Q8. Kotlin est le langage officiel pour :

A) iOS B) Android C) Windows D) Linux kernel

Q9. ?. en Kotlin s'appelle :

A) Elvis operator B) Safe call operator C) Force unwrap D) Smart cast

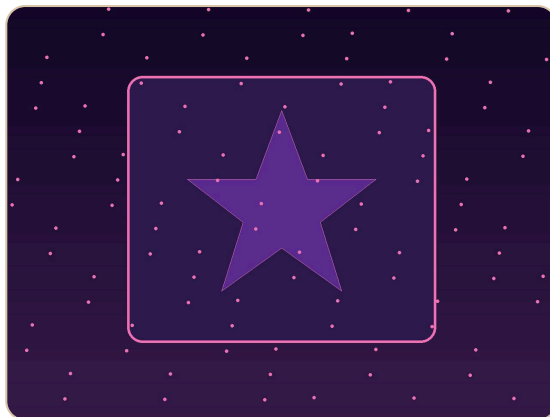
Q10. Result<T> sert a :

A) Stocker des listes B) Gerer succes/erreur sans exception C) Creer des threads D) Formater du texte

Reponses

1-B, 2-B, 3-B, 4-C, 5-B, 6-C, 7-B, 8-B, 9-B, 10-B

> **Astuce DanielCraft** - 8/10 ou plus ? Tu es pret pour Kotlin intermediaire.

Bravo !

Bravo. Tu codes en Kotlin.

Tu as termine Kotlin - Les bases

Félicitations. Tu sais écrire des programmes Kotlin concis, sûrs et idiomatiques. Tu maîtrises les variables, les types, les conditions, les boucles, les fonctions, la null safety, les classes, les data classes, les collections et la gestion d'erreurs.

Ce que tu as construit

- Un premier programme compile.
- Des fonctions avec null safety.
- Un gestionnaire de liste de courses CLI.
- Des exercices sur chaque notion.

La suite avec DanielCraft

Le livre **Kotlin - Intermediaire** couvrira :

- Coroutines et async/await.
- Jetpack Compose pour Android.
- Spring Boot avec Kotlin.
- Kotlin Multiplatform (KMP).
- Tests avancés et architecture.

> **Astuce DanielCraft** - Kotlin rend le code agréable à écrire. Choisis un petit projet Android ou CLI et pousse-le.

Un dernier conseil

Kotlin te rend plus productif dans tous les écosystèmes JVM. Comprendre null safety, data classes et les lambdas change ta façon de coder. Chaque programme qui compile est une victoire.

Tu as les bases. Maintenant, code.

Tu as les briques. Maintenant, code.